

Alke Wallet

Evolucion del modelo de datos: Received, Approach y Scalable

Este documento reúne las tres etapas de evolucion del modelo de datos de Alke Wallet, desde el modelo recibido originalmente, pasando por una mejora incremental (Approach), hasta una version normalizada en Tercera Forma Normal (Scalable). Cada seccion incluye el modelo conceptual, el modelo logico, el script SQL correspondiente y una comparativa con la version anterior.

Contenido

Modelo Received (version inicial)..... 3

Modelo Conceptual..... 3

1. Descripción general ¡Error! Marcador no definido.

2. 1.1 Modelo conceptual — entidades y atributos 3

 Usuario..... 3

 Moneda 3

 Transacción 4

3. 1.2 Modelo conceptual — entidades y atributos ¡Error! Marcador no definido.

1.2 Modelo logico — script SQL 6

1.3 Limitaciones detectadas 7

2. Modelo Approach (mejora incremental)..... 8

2.1 Modelo conceptual — entidades y atributos 8

 User..... 8

 Currency 8

 Transaction 8

2.2 Modelo logico — script SQL 8

2.3 Comparativa Received -> Approach..... 9

2.4 Limitaciones pendientes (no resueltas en Approach) 9

3. Modelo Scalable (3FN)..... 10

3.1 Modelo conceptual — entidades y atributos 10

 User (ya no almacena saldo)..... 10

 Currency 10

 Account (nueva entidad — saldo por usuario y moneda)..... 10

 Transaction (entre cuentas, no entre usuarios) 10

3.2 Modelo logico — script SQL 10

3.3 Comparativa Approach -> Scalable..... 11

3.4 Consulta objetivo resuelta 11

3.5 Limitaciones pendientes (fuera de alcance) 12

4. Conclusion general..... 12

Modelo Received (version inicial)

Modelo de base de datos original entregado como punto de partida del proyecto **AlkeWallet**.

Modelo Conceptual

Modelo conceptual inicial recibido para el sistema **AlkeWallet**, identificando las entidades, atributos, relaciones y las principales decisiones de diseño. Sirve como base para comprender las limitaciones del modelo original y las mejoras propuestas en versiones posteriores.

Propósito del modelo

El modelo busca representar un sistema de wallet digital donde los usuarios pueden realizar transacciones financieras entre sí. La versión inicial se enfoca en la gestión básica de usuarios, monedas y movimientos, sin establecer relaciones complejas entre las entidades.

Entidades y atributos

Entidad	Descripción	Atributos clave
Usuario	Usuarios registrados en el sistema.	user_id (PK), nombre, correo_electronico, contraseña, saldo
Moneda	Catálogo de divisas admitidas.	currency_id (PK), currency_name, currency_symbol
Transacción	Registro de transferencias entre usuarios.	transaction_id (PK), importe, transaction_date, receiver_user_id (FK), sender_user_id (FK)

Modelo conceptual — entidades y atributos

Usuario

Representa a cada usuario registrado en el sistema.

Atributo	Tipo	Restriccion	Descripcion
user_id	INT	PK	Identificador único del usuario
nombre	VARCHAR(150)	NULL	Nombre completo
correo_electronico	VARCHAR(150)	NULL	Correo (sin UNIQUE en este modelo)
contrasena	VARCHAR(255)	NULL	Hash de la contraseña
saldo	INT	NULL	Saldo en la moneda base (sin decimales)

Moneda

Representa las divisas disponibles en el monedero virtual.

Atributo	Tipo	Restriccion	Descripcion
currency_id	INT	PK	Identificador unico de la moneda
currency_name	VARCHAR(50)	NULL	Nombre de la moneda
currency_symbol	VARCHAR(50)	NULL	Simbolo monetario

Transacción

Registra las transferencias de dinero entre usuarios.

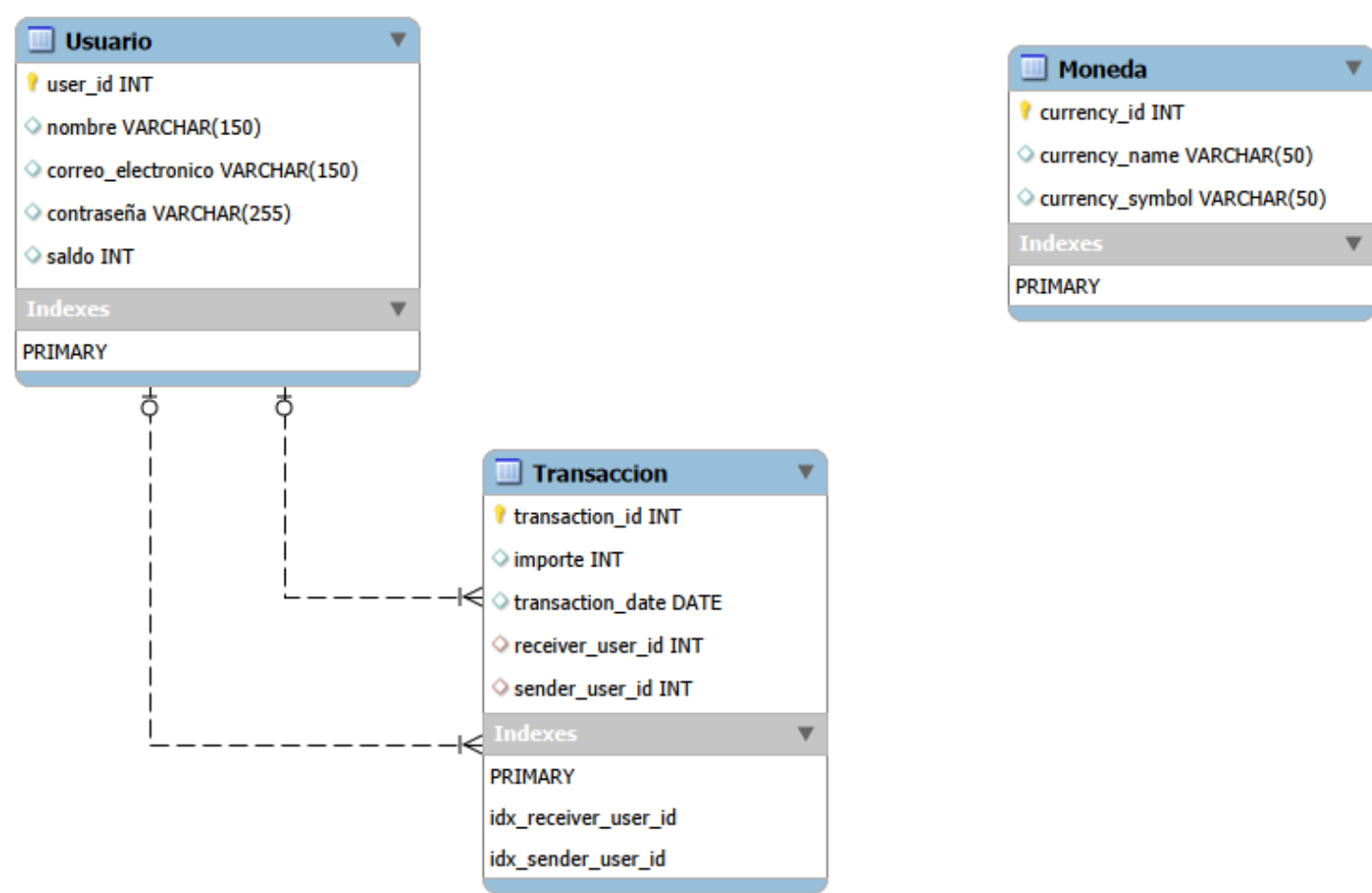
Atributo	Tipo	Restriccion	Descripcion
transaction_id	INT	PK	Identificador unico
importe	INT	NULL	Monto transferido (sin decimales)
transaction_date	DATE	NULL	Fecha de la operacion
receiver_user_id	INT	FK -> Usuario, NULL	Usuario receptor
sender_user_id	INT	FK -> Usuario, NULL	Usuario emisor

La tabla Moneda no tiene ninguna relacion con Usuario ni Transaccion: queda como catalogo aislado.

Relaciones entre entidades

Relación	Cardinalidad	PK/FK	Descripción
Usuario → Transacción (sender)	1 : N	sender_user_id → user_id	Un usuario puede enviar muchas transacciones. Cada transacción tiene un único emisor.
Usuario → Transacción (receiver)	1 : N	receiver_user_id → user_id	Un usuario puede recibir muchas transacciones. Cada transacción tiene un único receptor.
Moneda → (ninguna)	—	—	La entidad Moneda está desconectada: no se relaciona con Usuario ni Transacción.

Diagrama Entidad Relación



Limitaciones y decisiones de diseño

A continuación, se enumeran las principales decisiones de diseño adoptadas en el modelo original y sus implicaciones:

Decisión	Implicación / Limitación
Moneda como tabla aislada	No es posible asociar una transacción a una divisa concreta, ni conocer la moneda preferida de un usuario.
Saldo e importe como INT	No se admiten valores decimales (centavos), lo que impide representar montos como \$12.50 o \$0.99.
Campos NULL	Permite la creación de registros incompletos (ej. un usuario sin nombre).
Sin AUTO_INCREMENT en PKs	Los identificadores deben asignarse manualmente al insertar datos, aumentando el riesgo de errores.
ON DELETE NO ACTION	Si se elimina un usuario, las transacciones asociadas quedan huérfanas (referencias rotas).
Sin índices adicionales	Las consultas frecuentes por sender_user_id o receiver_user_id no estarán optimizadas.

```
CREATE SCHEMA IF NOT EXISTS `AlkeWallet` DEFAULT CHARACTER SET utf8 COLLATE utf8_bin ;
USE `AlkeWallet` ;

-----
-- Table `AlkeWallet`.`Usuario`
-----

DROP TABLE IF EXISTS `AlkeWallet`.`Usuario` ;

CREATE TABLE IF NOT EXISTS `AlkeWallet`.`Usuario` (
  `user_id` INT NOT NULL,
  `nombre` VARCHAR(150) NULL,
  `correo_electronico` VARCHAR(150) NULL,
  `contraseña` VARCHAR(255) NULL,
  `saldo` INT NULL,
  PRIMARY KEY (`user_id`))
ENGINE = InnoDB;

-----
-- Table `AlkeWallet`.`Moneda`
-----

DROP TABLE IF EXISTS `AlkeWallet`.`Moneda` ;

CREATE TABLE IF NOT EXISTS `AlkeWallet`.`Moneda` (
  `currency_id` INT NOT NULL,
  `currency_name` VARCHAR(50) NULL,
  `currency_symbol` VARCHAR(50) NULL,
  PRIMARY KEY (`currency_id`))
ENGINE = InnoDB;

-----
-- Table `AlkeWallet`.`Transaccion`
-----

DROP TABLE IF EXISTS `AlkeWallet`.`Transaccion` ;

CREATE TABLE IF NOT EXISTS `AlkeWallet`.`Transaccion` (
  `transaction_id` INT NOT NULL,
  `importe` INT NULL,
  `transaction_date` DATE NULL,
  `receiver_user_id` INT NULL,
  `sender_user_id` INT NULL,
  PRIMARY KEY (`transaction_id`),
  CONSTRAINT `fk_transaction_receiver_user_id`
    FOREIGN KEY (`receiver_user_id`)
      REFERENCES `AlkeWallet`.`Usuario` (`user_id`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION,
  CONSTRAINT `fk_transaction_sender_user_id`
    FOREIGN KEY (`sender_user_id`)
      REFERENCES `AlkeWallet`.`Usuario` (`user_id`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION)
ENGINE = InnoDB;
```

1.3 Limitaciones detectadas

Este modelo fue recibido como punto de partida y presenta varias limitaciones que deben corregirse en versiones posteriores:

Área	Problema	Impacto
Tipos de datos	saldo e importe son INT	No permite valores decimales (centavos).
Integridad	Campos NULL permitidos en casi todas las columnas	Posibles registros incompletos.
Identificadores	PK sin AUTO_INCREMENT	Asignación manual de IDs propensa a errores.
Relaciones	Moneda desconectada de Usuario y Transaccion	No se puede asociar una transacción a una divisa ni saber la moneda preferida de un usuario.
Acción referencial	ON DELETE NO ACTION	Eliminación de usuario deja transacciones huérfanas.
Rendimiento	Sin índices adicionales	Consultas por sender_user_id o receiver_user_id serán lentas con grandes volúmenes de datos.

2. Modelo Approach (mejora incremental)

Corrige las limitaciones del modelo Received sin modificar la esencia del esquema: se mantienen tres tablas principales, pero con tipos de datos correctos, restricciones de integridad, relacion con moneda e indices de rendimiento.

2.1 Modelo conceptual — entidades y atributos

User

Atributo	Tipo	Restriccion	Descripcion
user_id	INT	PK, AUTO_INCREMENT	Identificador unico
username	VARCHAR(100)	NOT NULL	Nombre completo
email	VARCHAR(150)	NOT NULL, UNIQUE	Correo unico
password	VARCHAR(255)	NOT NULL	Hash de la contrasena
current_balance	DECIMAL(15,2)	NOT NULL, DEFAULT 0	Saldo con decimales

Currency

Atributo	Tipo	Restriccion	Descripcion
currency_id	INT	PK, AUTO_INCREMENT	Identificador unico
currency_name	VARCHAR(50)	NOT NULL, UNIQUE	Nombre de la moneda
currency_symbol	VARCHAR(5)	NOT NULL, UNIQUE	Simbolo monetario

Transaction

Atributo	Tipo	Restriccion	Descripcion
transaction_id	INT	PK, AUTO_INCREMENT	Identificador unico
importe	DECIMAL(15,2)	NOT NULL	Monto con decimales
transaction_date	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Fecha y hora exacta
sender_user_id	INT	NOT NULL, FK -> User	Usuario emisor
receiver_user_id	INT	NOT NULL, FK -> User	Usuario receptor
currency_id	INT	NOT NULL, FK -> Currency	Moneda de la transaccion

2.2 Modelo logico — script SQL

```
CREATE DATABASE IF NOT EXISTS AlkeWallet;
USE AlkeWallet;

CREATE TABLE User (
  user_id    INT AUTO_INCREMENT PRIMARY KEY,
  username   VARCHAR(100) NOT NULL,
  email      VARCHAR(150) NOT NULL UNIQUE,
  password   VARCHAR(255) NOT NULL,
  current_balance DECIMAL(15,2) NOT NULL DEFAULT 0
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE Currency (
  currency_id  INT AUTO_INCREMENT PRIMARY KEY,
  currency_name VARCHAR(50) NOT NULL UNIQUE,
  currency_symbol VARCHAR(5) NOT NULL UNIQUE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE Transaction (
  transaction_id INT AUTO_INCREMENT PRIMARY KEY,
  importe        DECIMAL(15,2) NOT NULL,
  transaction_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  sender_user_id INT NOT NULL,
  receiver_user_id INT NOT NULL,
  currency_id    INT NOT NULL,
  CONSTRAINT fk_transaction_sender
    FOREIGN KEY (sender_user_id) REFERENCES User(user_id)
    ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT fk_transaction_receiver
    FOREIGN KEY (receiver_user_id) REFERENCES User(user_id)
    ON DELETE CASCADE ON UPDATE CASCADE,
```



```
CONSTRAINT fk_transaction_currency
  FOREIGN KEY (currency_id) REFERENCES Currency(currency_id)
  ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
CREATE INDEX idx_transaction_sender  ON Transaction (sender_user_id);
CREATE INDEX idx_transaction_receiver ON Transaction (receiver_user_id);
CREATE INDEX idx_transaction_currency ON Transaction (currency_id);
CREATE INDEX idx_transaction_date    ON Transaction (transaction_date);
```

2.3 Comparativa Received -> Approach

Aspecto	Received	Approach
Identificadores (PK)	INT NOT NULL (manual)	INT AUTO_INCREMENT
Campos monetarios	INT (sin decimales)	DECIMAL(15,2)
Campos obligatorios	NULL permitido casi todo	NOT NULL en todos los campos
Unicidad	No definida	UNIQUE en email, currency_name, currency_symbol
Relacion con moneda	Moneda aislada	currency_id (FK) en Transaction
Fecha	DATE (solo dia)	DATETIME (dia + hora)
Indices	Solo PK implicitas	Indices explicitos en FKs y fecha
Accion referencial	NO ACTION	ON DELETE/UPDATE CASCADE

2.4 Limitaciones pendientes (no resueltas en Approach)

Limitacion	Impacto	Solucion en Scalable
Saldo unico en User	No soporta multiples divisas por usuario	Extraer saldo a tabla Account
Sin restricciones CHECK	No valida importe > 0 ni balance >= 0	Agregar CHECK constraints
CASCADE en produccion	Puede borrar historial accidentalmente	Cambiar a RESTRICT o soft delete

3. Modelo Scalable (3FN)

Version normalizada del esquema, que introduce la entidad Account para soportar multiples saldos por usuario (multi-moneda), agrega restricciones CHECK y protege el historial financiero con ON DELETE RESTRICT.

3.1 Modelo conceptual — entidades y atributos

User (ya no almacena saldo)

Atributo	Tipo	Restriccion	Descripcion
user_id	INT	PK, AUTO_INCREMENT	Identificador unico
user_name	VARCHAR(150)	NOT NULL	Nombre completo
email	VARCHAR(150)	NOT NULL, UNIQUE	Correo unico
password	VARCHAR(255)	NOT NULL	Hash de la contraseña

Currency

Atributo	Tipo	Restriccion	Descripcion
currency_id	INT	PK, AUTO_INCREMENT	Identificador unico
currency_name	VARCHAR(50)	NOT NULL, UNIQUE	Nombre de la moneda
currency_symbol	VARCHAR(5)	NOT NULL, UNIQUE	Simbolo monetario

Account (nueva entidad — saldo por usuario y moneda)

Atributo	Tipo	Restriccion	Descripcion
account_id	INT	PK, AUTO_INCREMENT	Identificador unico de la cuenta
user_id	INT	NOT NULL, FK -> User	Usuario propietario
currency_id	INT	NOT NULL, FK -> Currency	Moneda de la cuenta
current_balance	DECIMAL(15,2)	NOT NULL, DEFAULT 0, CHECK >= 0	Saldo en esa moneda
is_default	TINYINT(1)	NOT NULL, DEFAULT 0	Cuenta principal del usuario

Transaction (entre cuentas, no entre usuarios)

Atributo	Tipo	Restriccion	Descripcion
transaction_id	INT	PK, AUTO_INCREMENT	Identificador unico
importe	DECIMAL(15,2)	NOT NULL, CHECK > 0	Monto transferido
transaction_date	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Fecha y hora
sender_account_id	INT	NOT NULL, FK -> Account	Cuenta emisora
receive_account_id	INT	NOT NULL, FK -> Account	Cuenta receptora

Regla de negocio: UNIQUE(user_id, currency_id) en Account evita dos cuentas del mismo usuario en la misma moneda. Solo una cuenta por usuario puede tener is_default = 1; en MySQL esto se aplica con triggers (ver script), ya que MySQL no soporta indices unicos parciales (WHERE dentro de CREATE INDEX).

3.2 Modelo logico — script SQL

```
CREATE DATABASE IF NOT EXISTS AlkeWallet;
USE AlkeWallet;

CREATE TABLE User (
  user_id INT AUTO_INCREMENT PRIMARY KEY,
  user_name VARCHAR(150) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE Currency (
  currency_id INT AUTO_INCREMENT PRIMARY KEY,
  currency_name VARCHAR(50) NOT NULL UNIQUE,
```

```
currency_symbol VARCHAR(5) NOT NULL UNIQUE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE Account (
  account_id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  currency_id INT NOT NULL,
  current_balance DECIMAL(15,2) NOT NULL DEFAULT 0,
  is_default TINYINT(1) NOT NULL DEFAULT 0,
  CONSTRAINT chk_balance_no_negativo CHECK (current_balance >= 0),
  CONSTRAINT uq_account_user_currency UNIQUE (user_id, currency_id),
  CONSTRAINT fk_account_user
    FOREIGN KEY (user_id) REFERENCES User(user_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT,
  CONSTRAINT fk_account_currency
    FOREIGN KEY (currency_id) REFERENCES Currency(currency_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
CREATE TABLE Transaction (
  transaction_id INT AUTO_INCREMENT PRIMARY KEY,
  importe DECIMAL(15,2) NOT NULL,
  transaction_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  sender_account_id INT NOT NULL,
  receive_account_id INT NOT NULL,
  CONSTRAINT chk_importe_positivo CHECK (importe > 0),
  CONSTRAINT chk_cuentas_distintas CHECK (sender_account_id <> receive_account_id),
  CONSTRAINT fk_transaction_sender
    FOREIGN KEY (sender_account_id) REFERENCES Account(account_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT,
  CONSTRAINT fk_transaction_receiver
    FOREIGN KEY (receive_account_id) REFERENCES Account(account_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

-- MySQL no soporta indices unicos parciales; se usa un trigger
-- para garantizar una sola cuenta default por usuario.

```
DELIMITER //
CREATE TRIGGER trg_unico_default_insert
BEFORE INSERT ON Account
FOR EACH ROW
BEGIN
  IF NEW.is_default = 1 THEN
    UPDATE Account SET is_default = 0
    WHERE user_id = NEW.user_id AND is_default = 1;
  END IF;
END //
DELIMITER ;
```

3.3 Comparativa Approach -> Scalable

Aspecto	Approach	Scalable
Saldo del usuario	En User.current_balance (unico)	En Account.current_balance (uno por moneda)
Relacion User-Currency	Indirecta (via Transaction)	Directa (via Account)
Transacciones	Entre usuarios	Entre cuentas
Accion referencial	ON DELETE CASCADE	ON DELETE RESTRICT
Validaciones de negocio	Sin CHECK	CHECK balance >= 0, importe > 0, sender <> receiver
Cuenta predeterminada	No existia	is_default en Account
Normalizacion	2FN	3FN

3.4 Consulta objetivo resuelta

Con este modelo, la consulta "nombre de la moneda elegida por un usuario especifico" se resuelve de forma directa, sin necesidad de inferir a partir del historial de transacciones:

```
SELECT u.user_id, u.user_name, c.currency_name
FROM User u
```

```
JOIN Account a ON u.user_id = a.user_id AND a.is_default = 1
JOIN Currency c ON a.currency_id = c.currency_id
WHERE u.user_id = 1;
```

3.5 Limitaciones pendientes (fuera de alcance)

Limitacion	Detalle
Conversion de monedas	No incluye tasas de cambio ni conversion automatica
Auditoria avanzada	No registra quien/modifico cuando (requeriria triggers o tabla de auditoria)
Borrado logico	No se implementa is_active en User

4. Conclusion general

La evolucion Received -> Approach -> Scalable documenta de forma progresiva como un modelo de datos puede pasar de un esquema funcional pero con riesgos de integridad, a una version production-ready con tipos de datos correctos y relaciones completas (Approach), y finalmente a una version normalizada en 3FN que soporta multiples monedas por usuario y protege el historial financiero de borrados accidentales (Scalable). Cada version es independiente y ejecutable; la eleccion entre ellas depende de los requisitos reales del sistema.